

CS 204

Advanced Programming

(Week 1: Introduction)

Kamer Kaya

Faculty of Engineering and Natural Sciences

Sabanci University

kaya@sabanciuniv.edu

Outline

- Course info
- Good programming practice(s) – style, commenting
- Preprocessor, compiler, linker...
- Debugging
 - using preprocessor directives
 - *assert* statement

CS204 in a Nutshell

- Prerequisite (only for SU students): CS201
- Coverage (Tentative)
 - Introduction to data structures
 - Pointers and dynamic memory allocation
 - Linked lists, stacks, and queues
 - Advanced structured and object-oriented programming
 - Templates, templated classes and functions
 - Data representation, bitwise operations
 - Advanced issues on classes and object-oriented programming
 - Inheritance, polymorphism and advanced object-oriented design
 - Writing better, more efficient programs
 - Preprocessing and compiler directives
 - Reusable software (using/creating libraries)
 - Exception handling
 - (Concurrent) programming with threads
 - Visual programming and graphical user interface with QT

CS204 in a Nutshell

- Text Book
 - Main text is "Ivor Horton's Beginning Visual C++ 201X", by Ivor Horton, ISBN: 978-1-118-36808-4.
 - You are responsible material covered in class too.
 - No rush to buy the book
- Grading
 - Tentative - Midterm1: 25% ; Midterm2: 30% ; Final: 45%
 - There will be 5-6 homeworks; the HWs will not be graded

Web

- The SUCourse web page contains the necessary UPDATED information and syllabus. Please follow the announcements...
- Lecture notes will be posted weekly (please try to go over the slides once before the course)
- We will use **SUCourse** to submit/collect homeworks and many other things such as discussions and announcements.
- We will use **SUCourse** to answer your questions.

Schedule, Labs and TAs

- Syllabus will be at SUCourse.
- TAs will go over course material from the past week through some examples and explain that week's homework (if there is one) in the lab time. So, this is half recitation, half lab hour.
- You can ask questions in the Lab hours.
- We also have MANY (everyday, multiple hours) office hours for the questions to be answered.
 - **USE THEM**

Make-up Policy

- All medical reports must be submitted and all excuses must be reported BEFORE the exam.
- **It is at the discretion of the instructor to give a make-up exam**
 - LETTER FOR SABANCI UNIVERSITY UNDERGRADUATE PROGRAMS, Clause 19/4

"A make-up exam is given to students who cannot take an exam for reasons **acceptable to the faculty member.**"
 - **Do not rely on fake medical reports**
- The make-up exams will be given as written exams
- The make-up exams can be at anytime (including the whole final period)
- The make-up exams will be **harder** than regular ones for fairness reasons
 - **Only** because, make-up takers will have more time to study

Good Programming Practices, Style, Debugging

We expect you to follow the style/practice guidelines
listed here for all future homeworks/projects

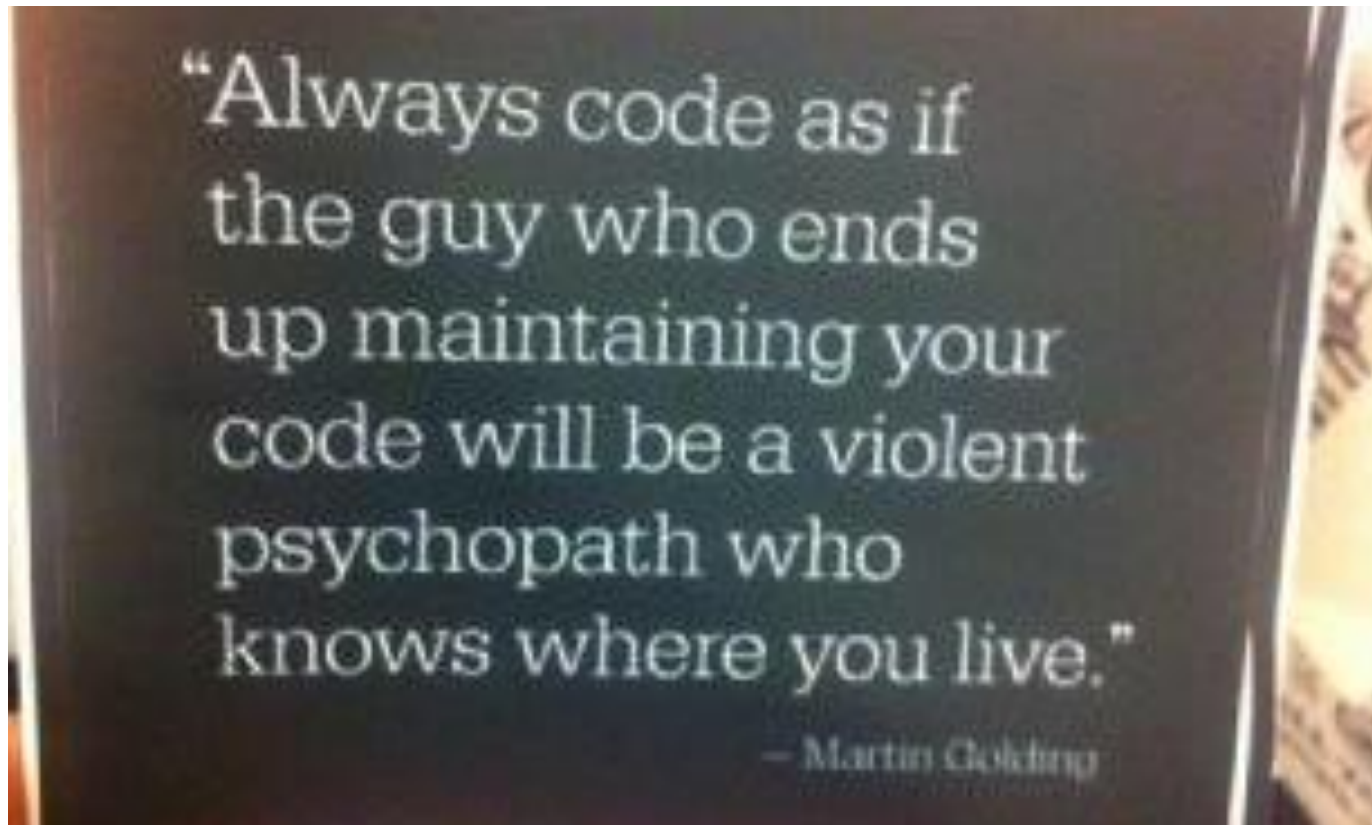
Programming

- ... The programmer, like the poet, works only slightly removed from pure thought-stuff. He builds his castles in the air, from air, creating by exertion of the imagination. The program construct, unlike the poet's words, is real in the sense that it moves and works, producing visible outputs separately from the construct itself. One types the correct incantation on a keyboard, and a display screen comes to life, showing things that never were nor could be.

*The Mythical Man-Month: Essays on Software Engineering
Anniversary Edition, by Frederick P. Brooks, Jr., Addison-Wesley © 1995.*

Programming

- Time spent as a software engineer will be spent mostly on maintaining and improving/upgrading code:
 - 80-90% of yours or others' time
- # of lines per major application:
 - May go as much as >1 billion!
 - <http://www.informationisbeautiful.net/visualizations/million-lines-of-code/>
 - It has to be written well so that upgrading/maintaining is manageable
- Some old programs cannot be improved because nobody understands the code!
 - If only all the programmers care about documentation and reusability



See <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>
for core C++ guidelines

Good Programming Habits

- Develop and test your code incrementally
 - Code defensively
 - Code clearly
 - Use modular code
- Include debugging code/tests
 - Include comments
 - ...

Good Programming Habits: Comments

- A set of thoughts:
<https://www.nickmccullum.com/how-to-write-cpp-comments/>
- How much?
 - Enough for someone to be able to understand/modify your code in the future
 - Ideally when the comments are read without looking at the code, everything should be clear

Good Programming Habits: Comments

- Where?
 - **program header**: describe purpose, usage, inputs, outputs, known bugs, author, date, version history, external references
 - **each function and major part**: purpose, input and output parameters, returning value
 - **each variable**: describe the purpose and usage
 - **instructions**: not as paraphrasing the code, describe semantically.

Good Programming Habits: Comments

- When?
 - Comment when first writing
 - then update as necessary
 - Check the comments before finalizing
- You must use sufficient comments in each of your homework submissions
 - penalty: up to 50% in a homework

Good Programming Habits: Comments

- **Style**
 - There are different conventions. Consistency is important.
 - **A suggestion:** comments and codes should be separable

Good Programming Habits: Comments

- What does this code fragment do?

```
temp = box1.x;  
box1.x = box2.x;  
box2.x = temp;  
temp = box1.y;  
box1.y = box2.y;  
box2.y = temp;
```

Good Programming Habits: Comments

- What does this code fragment do?

```
/* swap the two corners */
```

```
/* swap the x coordinate */
```

```
temp = box1.x;
```

```
box1.x = box2.x;
```

```
box2.x = temp;
```

```
/* swap the y coordinate */
```

```
temp = box1.y;
```

```
box1.y = box2.y;
```

```
box2.y = temp;
```

Good Programming Habits: Naming Conventions

- Naming Style suggestions:

```
balanceOwedToMe    //first word lowercase, then capitalized initials
                   //good for normal vars
RecordsInFile      //capitalized initials, good for global variables
MAX_STUDENT_NUM    //all caps - good for constants
```

- e.g. `const short MAX_STUDENT_NUM = 1000;`

- See <http://geosoft.no/development/cppstyle.html>
for a very structured convention that is used by many professionals.

Good Programming Habits: Naming Conventions

- No dummy names such as **dummy**, **myvar**, **robinVanPersie**, **a**, **b**, **c**, **x**, **y**, **z**
- Use **context specific** and **self-explanatory** names
 - This improves the readability of your code
- Similarly, do not call your homeworks **deneme**, **test** etc. 😊

Good Programming Habits: Indentation

- Vertical alignment of the programming blocks
 - Beginning and end of the blocks should be distinguishable
- Use and location of the curly brackets { }
- It is a good idea to always use it **even for one-line blocks**
- Two schools. Whatever you choose, adopt and stick to one format

```
for (i=0; i < NUMCLASS; i++) {  
    ...  
}
```

or

```
for (i=0; i < NUMCLASS; i++)  
{  
    ...  
}
```

Good Programming Habits: Clarity

- Use empty spaces in your code to separate logical blocks
 - As you make paragraphs in essays
- Do not code for compactness
 - We are not encoding knowledge
 - Clarity is very important

```
o = 5;  
o = --o + o-- ;//Use ++ or -- only, on one line. What is o?
```

```
value = 1;  
res = (value++ * 5) + (value++ * 3); //What is res?  
// do not even do this, even though it looks a little  
more innocent
```

- -- **C++**: At certain specified points in the execution sequence called **sequence points**, all *side effects* of previous evaluations shall be complete and no *side effects* of subsequent evaluations shall have taken place. (§ 1.9/7)

Good Programming Habits: Clarity

- Coding for efficiency is usually important: **Efficient programming is to move out unnecessary code or to avoid unnecessary iterations, so that no extra computation is done.**
- Do not confuse compactness with programming efficiency.

Compactness refers to how short your code is. You don't want to make your program unnecessarily long.

But compactness must not yield cryptic code: rule is to put only as much code on one line as it is still clearly understandable.

Good Programming Habits: Code Development

- Develop and test incrementally
 - Rather than writing the whole program first
 - one of the biggest source of frustration when trying to start a project or homework.
- Code modularly
 - Think the solution in pieces
 - Without losing their connection to the big picture
- Code defensively
 - Robustness
 - Proactive defense
 - Take precautions to all possible mistakes
 - Do not trust yourself
 - Anybody can make mistakes
 - Especially do not trust the user of your program; he/she can do anything!

Example: Check if a file is opened successfully or not. Otherwise, you can get a crash or I/O operations do not work out.

```
ifstream input(filename.c_str());
if (input.fail()) {
    cout << "could not open file " << filename << endl;
    return 0;
}
```

Good Programming Habits:

Code development

- Other's code
 - You will often have to deal with!
 - **Add comments as you understand the code, for your sake and others in the future.**



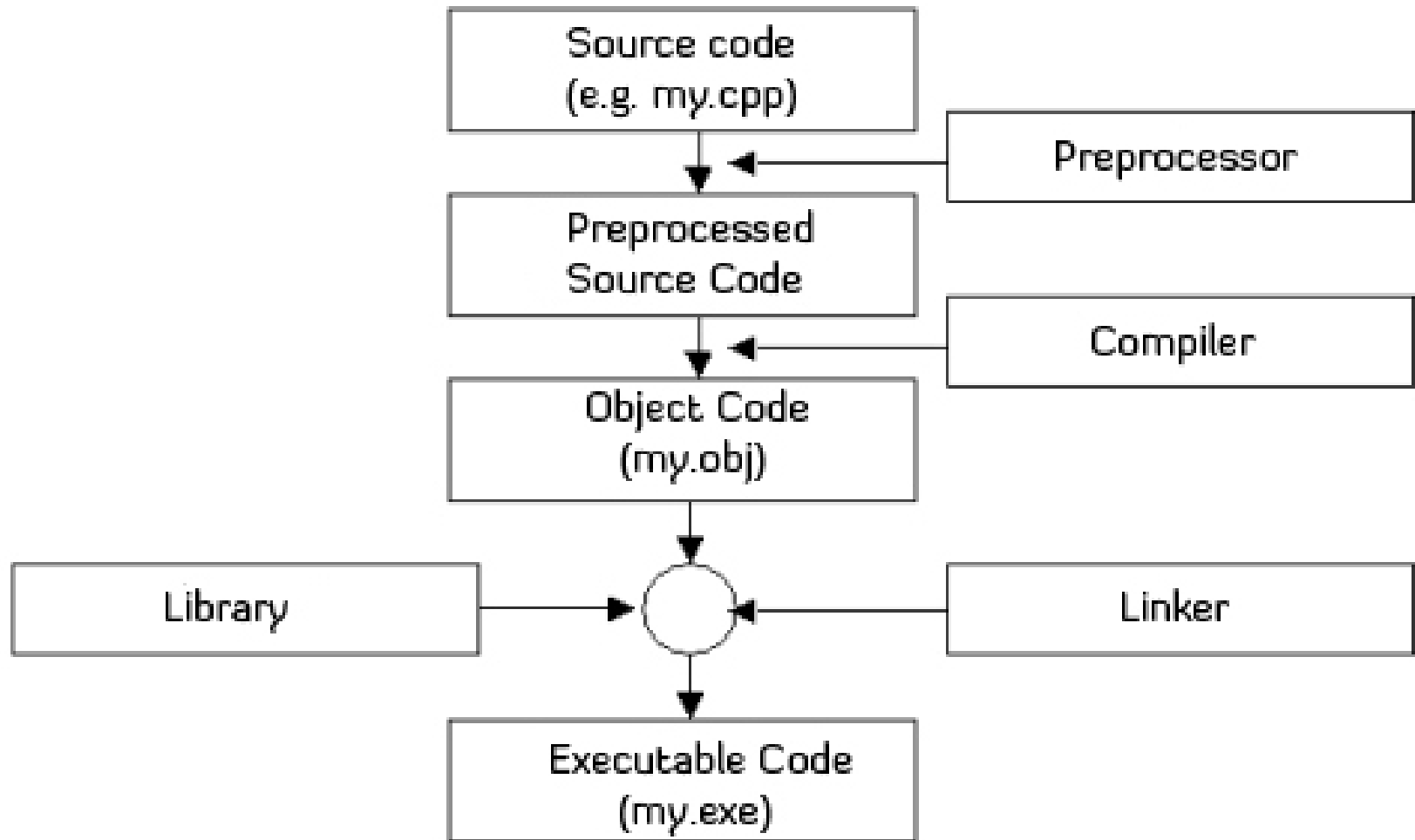
Good Programming Habits:

Code development

- Don't ignore warnings
 - Try to fix or at least understand why they exist.
 - They may be precursors of catastrophes
 - For example
 - `int j` – unreferenced variable
 - Maybe you wanted to use `j`, but instead of `j` you wrote `jj` and `jj` is another variable (**we should not use `jj` as a variable name**).
 - In ideal case, you should see no warnings when you compile
 - You can change your project's warning settings (see Warning levels under Project → Properties → C/C++) so that more important warnings do not get overlooked in a flood of warnings
 - Of course, with the risk of missing some potential problems.

Preprocessor, Compiler, Linker

Preprocessor, Compiler, Linker



Preprocessor

- A preprocessor is a program that takes your source file and outputs an intermediate file (**translation unit**) before compilation.
- It does text substitutions controlled by preprocessor directives, which are lines starting with **#** character

```
#include "filename"
```

```
#include <filename>
```

Ex: `#include "date.h"`

- contents of the include file (**date.h** in the example) is inserted into the translation unit
 - The process is recursive such that if a `#include` file has other `#include` files inside, they are also inserted.
- Full path may be given between " and "
- Difference between " " and < >
 - If quotes are used, first, the folder of the file containing the include statement is checked.
- Caution: **You have to avoid multiple inclusions of the same file** in different include files.
 - Using *if* family compiler directives (will see later)

#define

`#define identifier token-string`

- to define a preprocessor identifier
- All occurrences of *identifier* are replaced by *token-string* before compilation
- commonly used to define constants (but const definitions can also be used; it is up to you)

```
#define PI 3.1416
```

```
#define MAXSIZE 10000
```

```
#define PROMPT "Please enter your move: "
```

#define

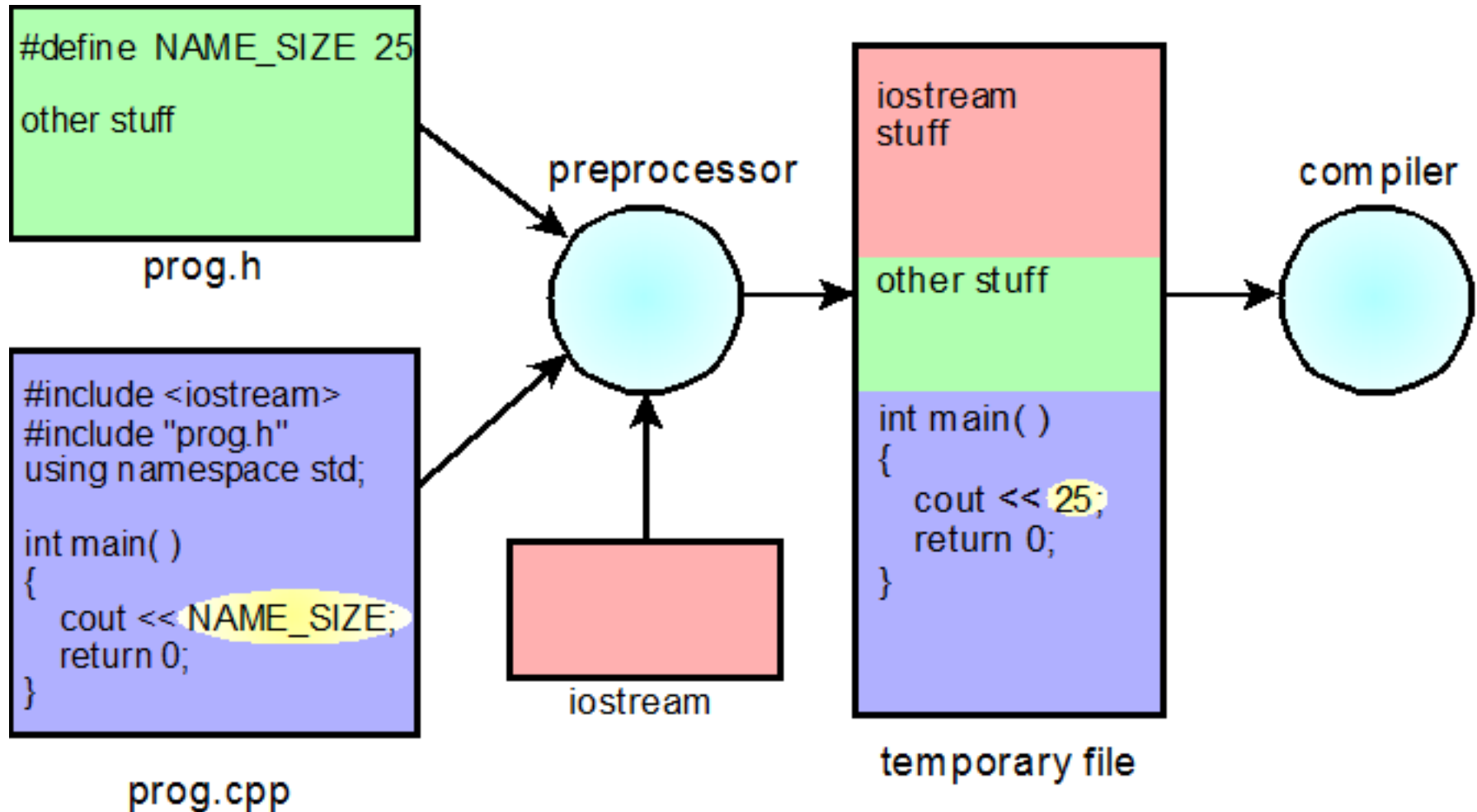
As a good programming practice, prefer `const` to `#define`:

```
const double PI = 3.1416;
```

- Code needs to be clear also for your fellow coders. If `#define MAXSIZE` has been used and if there is an error in the use of the constant the error message won't refer to `MAXSIZE`.
 - In case you are not the original coder, you won't have a clue where 10,000 comes from in the error message (also in some debuggers you won't see `MAXSIZE`).
- Furthermore, more copies of 3.1416 are created with **`#define`**

A more general rule is: “prefer compiler to preprocessor”

Preprocessor: until now



Conditional Preprocessing

```
#define identifier  
#ifdef identifier  
    statements  
#endif
```

- *statements* are included translation unit if *identifier* is defined

```
#ifndef identifier  
    statements  
#endif
```

- *statements* are included translation unit if *identifier* is **not** defined
- In both cases, for *identifiers* defined using `#define`
- regular identifiers do not count

Conditional Preprocessing

Actual cpp file

```
#include "dice.h"
#define MYDEBUG

#ifdef MYDEBUG
    cout << n;
#endif

DoSmtHg();
```



Translation Unit

```
Class dice
...
...
}

cout << n;

DoSmtHg();
```

#undef

`#undef identifier`

- Removes *identifier* which is previously created with `#define`
- In other words, it becomes undefined
- Only affects subsequent occurrences of *identifier*
 - not the previous ones
- CAUTION FOR Visual Studio:
 - If you use an undefined identifier in a preprocessor expression, it is considered as zero, rather than a compiler error in MS Visual Studio. However, using an undefined identifier in a normal statement yields a compiler error. A defined one won't.

Conditional Preprocessing

- More general conditional preprocessing can be performed using `#if`, `#elif`, `#else`, `#endif` preprocessor directives
 - Main idea is similar to nested if statements (but this is done at preprocessing stage)

`#if` *constant-expression*
 statements

`#elif` *constant-expression*
 statements

`#else`
 statements

`#endif`

- } - Optional,
- } - Can have multiple times
- } - Optional

- Constant expression must include only `#defined` identifiers and literals
 - Comparison and boolean operators can be used
- Do not use regular variables here. In VC++, they are considered as 0 no matter their values are. Thus it causes unexpected results
- See [testdefine.cpp](#) for an example

Macros

- The `#define` directive is also used to define macros with parameters:

```
#define MULTIPLY(a,b) a*b  
#define SQUARE(x) x*x
```

- After these macro definitions, the preprocessor will replace any text in the forms of

```
MULTIPLY (something, anotherthing) with  
something*anotherthing
```

```
SQUARE (something) with something*something
```

- These macros are supposed to define an operation without writing a real C++ function. They seem promising since
 - They do not care about the variable types (i.e., you can use `SQUARE (x)` when `x` is `double` or `int`) .
 - You can avoid the overhead of function calls.
- Still, they are not usually recommended (See next)

Use of Macros is not recommended

- Consider the following code

```
#define SQUARE(x) x*x
int x = SQUARE(3+5);
int y = 100/SQUARE(5);
int k=5;
int z = SQUARE(++k);
```

- What is the value of x?
 - Answer: 23, why?
- What is the value of y?
 - Answer: 100, why?
- What is the value of z?
 - Answer: 49 – undefined behaviour, why?
- More correct way of defining this macro is

```
#define SQUARE(x) ((x)*(x))
```

- This corrects first two cases above. But even this does not solve the third problem.

Use of Macros is not recommended

- Consider the following code

```
#define DOUBLE_INCREMENT(a,b) a++; b++;  
int x = 0, y = 0, z = 0;  
if(z == 1)  
    DOUBLE_INCREMENT(x, y);
```

- What is the value of x and y?
 - Answer: 0 and 1, why?
- NOTE: With or without macros, always use curly brackets for `if`, `for`, `while`, etc. clauses as mentioned before.

Use functions instead of Macros

- Use functions instead

```
int Square (int x) {  
    return x*x;  
}
```

- To handle multiple types and function call overheads, use

```
template<typename T>  
inline T square(T x) {  
    return x * x;  
}
```

- We will look at to templates later.

Compiler & Linker

- The input to the compiler is the **translation unit** generated by the preprocessor (starting from a single source file - e.g. main.cpp).
- The **compiler** generates an object file (e.g. main.obj). Sometimes object files have .o extension.
- If the project has several cpp files, normally they are compiled separately yielding different object files
- The **linker** combines all necessary object files and libraries in the project together to create an executable program.
 - More on the libraries later.

Debugging

Debugging: Diagnostic Messages

- If you are coding in VS, you have elegant and efficient embedded tools for debugging. And in Linux you have **gdb** and **valgrind** (and others). But `cout` is **sometimes** the best (the fastest) debugger
- To debug your program, you may include some diagnostic messages
- For example,

```
//See if all values are correct
for (i=0; i < 26; i++)
    cout << counter[i] << " ";
cout << endl;
```

Debugging with extra statements

- When you think you are done debugging, you can do one of the followings to turn debug statements off (so that your program runs fast and without debug messages)

1) remove all this debug code (e.g. cout's)

Too much work, plus you may need them again when some other problem shows up (if you need debugging later on)

2) make them conditional to a program variable. For instance:

```
if (debug == 1) {  
    for (i=0; i < 26; i++)  
        cout << counter[i];  
    cout << endl;  
}
```

- Then by setting the variable debug to 1 or 0 on top of the program, you can control whether the statements will be output or not.
- Tests such as `if (debug == 1)` are still done (slows the release version)!

Debugging with extra statements

- Best way: Using the preprocessor directive `_DEBUG` (Microsoft) and conditional preprocessing

```
#ifdef _DEBUG
    for (i=0; i < 26; i++)
        cout << counter[i];
#endif
```

- This code will only be compiled if the `_DEBUG` is defined
 - `_DEBUG` is a standard preprocessor identifier which is defined in the Debug configuration, but not defined in the Release configuration.
 - This is automatic when you switch between these two configurations
 - So when the preprocessor parses your code:
 - If the `_DEBUG` preprocessing identifier is defined, it will remove the `#ifdef` and `#endif`, but include the lines between in your program.
 - If the `_DEBUG` prep. identifier is NOT defined, the preprocessor will remove the `#ifdef` and `#endif` and the lines between them out of your program. So your program will be as if you have never written those lines (no extra tests done)

Debugging: Debug/Release

- You normally develop under Debug mode (a.k.a. Debug configuration)
 - After testing your program, you may want to submit it in release mode (release configuration)
- When you are developing with Debug option, the compiler keeps track of symbolic information so that you can watch over variable values, function call hierarchy etc., at every step in your code.
- Once you are done debugging and you will release the code to the customer (or teacher), you want to remove this information and any other debugging code you may have added so that your executable is cleaner, smaller and faster.

Preprocessor Output debug version

```
n = 1;  
  
#ifdef _DEBUG  
    cout << n;  
#endif  
  
DoSmthg();  
DoSmthgElse();
```

```
n = 1;  
  
cout << n;  
  
DoSmthg();  
DoSmthgElse();
```

_DEBUG is defined as a standard preprocessor identifier under the **Debug build of your code.**

Preprocessor Output: release version

```
n = 1;
```

```
#ifdef _DEBUG  
    cout << n;  
#endif
```

```
DoSmthg();  
DoSmthgElse();
```

```
n = 1;
```

```
DoSmthg();  
DoSmthgElse();
```

`_DEBUG` is **not defined** as a preprocessor identifier **under the release build of your code**.

Debugging

- You can also use your own preprocessor definitions:
 - You can either add them to the preprocessor definitions under Project/Properties/... where `_DEBUG` was defined
 - Or you can define them with `#define` in the beginning of the file

```
//You can define or not define your debug flags:
#define SHORTDBG
//#define MAJORDBG
...

#ifdef SHORTDBG
    cout << "*** Value of num = " << num << endl;
#endif

#ifdef MAJORDBG
    cout << "*** Value of num = " << num << endl;
    cout << "*** Value of info = " << info << endl;
    cout << "*** Value of t = " << t << endl;
    ...
#endif
```

Debugging

- Previous solutions require to comment out user defined preprocessor definitions for the release config.
- Alternatively, you can also make them *conditioned* on the automatic flag `_DEBUG`

```
#ifdef _DEBUG
    #define SHORTDBG          //this code won't be compiled under release config
    // #define MAJORDBG        //this code won't be compiled under release config
#endif
```

- This way, if you are running under the release config., you don't need to turn `SHORTDBG` or `MAJORDBG` off. Since their definition won't even be compiled, your flags won't be defined.
- See [debugging.cpp](#) for an example

Debugging: Assert statements

- Syntax:

```
assert (boolean-expression);
```

- If the *boolean-expression* evaluates to true, execution continues.
- If the *boolean-expression* evaluates to false, execution is halted (program aborts)
 - A message is dumped at this point, something like the following

```
assertion failed: i < j   file code.cpp, line 543
```

- You have to have `#include <cassert>` at the beginning of the program to use assert
- Use assert statements to check for conditions that should be true

```
#include <cassert>
...
assert (p != nullptr);
...

assert ( i < j );
```

Debugging: Assert statements

- assert statement **should not** replace checking and **handling** errors (e.g., if the file is found or not)
- provides compact checks for things that should not happen (e.g. would cause the program crash)
 - Exceptions
- Other alternative methods?
 - We will see *exception handling* in the coming weeks for a neat organization of error handling

Debugging: Assert statements

- The assert statement can be "turned off" by defining the preprocessor identifier `NDEBUG` before including `cassert`

```
#define NDEBUG                // No Debug
#include <cassert>
```

- If you use the release configuration, `NDEBUG` is already defined, so you do not need to define it yourself.
- In `cassert`, `NDEBUG` is checked if it is defined or not:
 - If it is defined, the `assert` statement evaluates to an **empty statement**, which is what we want for the Release configuration, so the tests are not done and do not slow the program.
 - If it is not defined (Debug mode/configuration), the `assert` statements stay and do the tests as intended.

Preprocessor, Compiler, Linker ctd.

Header Guards

- A very important use of conditional compilation is to prevent multiple inclusion of header files
 - Multiple inclusion would duplicate declarations, resulting in compiler errors.
 - Maybe you can manually check multiple inclusions of the same file in a single .cpp or .h, but when header files include themselves this manual check becomes very hard → so we need an automatic way for that.
 - Automatic way is to assign a preprocessor definition for each header file (convention is `_HEADERFILENAME`, where the dot is replaced by `_` as well) and check if it is defined before including the content of the header file. It is defined for the first time of inclusion, so in the next trials of inclusion, it is not going to be included.
- In the following example, `_DICE_H` is defined when "dice.h" is included the first time, hence later inclusions of "dice.h" won't have any effect on translation unit.

```
#ifndef    _DICE_H
#define    _DICE_H

...    //dice content is here

#endif
```

Compilation with Several Files

- Sometimes we may need to compile several cpp files
 - dice.cpp, randgen.cpp, prompt.cpp, ...
 - because some functions and class implementations are in those files
 - if you are using them in your program, you have to compile them together with your main cpp file
 - add to the same project
 - But there must be only one cpp file with main function
- Sometimes, the class and function implementations are in precompiled libraries
 - ready-to-use object code
 - need not to compile, but add the .lib file to the project

Linker

- Combines all necessary object files and libraries together to create an executable program.
- May create errors even if compilation is successful
 - unresolved external symbol ...
 - undefined Symbol ...
 - Primary reason would be missing libraries or implementation files (like date.cpp, prompt.cpp)
 - header files provide the definition, but body is missing!
 - If you have such errors, try to find where those symbols are implemented and include those libs or cpps in your project

Libraries

- Creating a statically linked library
 - choose your project as win32 console application but select "static library " as the application type
 - Add or write the necessary header and cpp files for your class and/or library
 - No main function
 - Build – the output is a .lib file
 - See [staticlib.h](#) and [staticlib.cpp](#) for an example
- Using this library
 - Create your project that will use this library
 - Add a main cpp file that will do the job by using this library
 - This is the cpp file with main function
 - Add the library (.lib file) as a resource file to the project
 - You do not need to add the cpp file that contains the library function/class implementation
 - Compile, link and run
 - See [testlib.cpp](#) for an example

Static versus Dynamic (shared) Libraries

- The libraries you link **statically** are **linked into the final executable** by the linker (remember that they are pre-compiled).
- **Dynamic libraries** are libraries that have the library name embedded into the executable but the library itself is **not linked** to the final executable.
 - Thus the size of executable is smaller
- When you build a program, the linker verifies that all the symbols (functions, variables ...) required by the program, are either linked into the program, or in one of its shared libraries. However, the object files from the dynamic library are not inserted into the executable file.
- Instead, when your program is **started**, a program in the system (called **dynamic loader**) checks out which shared (dynamic) libraries are needed for the program, loads them to memory, and attaches them to the copy of your program in memory.
 - Thus, launching your program is a bit –insignificantly – slower